

Department of Electronics and Communications
Engineering

Information Technology Institute (ITI)

K8s Demo Project

Prepared By: Ganna Hossam El-deen Mohamed

Contents

1	Introduction	2
2	Project Setup	2
3	Docker Image Build and Push	3
3.1	Initial Image Building	3
3.2	Final Image Build for Push	4
3.3	Pushing to Docker Hub	5
4	Docker Run Execution	6
5	Docker Compose Execution	6
6	Kubernetes Run Execution	7
6.1	Building Images for Minikube	7
6.2	Verifying for Minikube	7
7	Final Output	8
8	Architecture Diagram	8
9	Conclusion	9

1 Introduction

This Project documents the step-by-step building of a three-tier web application consisting of a Frontend, a Backend, and a PostgreSQL database. The application is deployed and tested across three different environments: manual Docker commands, Docker Compose, and a local Kubernetes cluster (Minikube). All custom images are built and published to Docker Hub.

2 Project Setup

The project is cloned and the directory structure is verified before starting.

```
IOT1@IOT-ITI:~$ mkdir project
IOT1@IOT-ITI:~$ cd project
IOT1@IOT-ITI:~/project$ git clone https://github.com/Es1am-Fawzi/k8s-demo.git
bash: git: command not found...
Install package 'git-core' to provide command 'git'? [N/y] y

* Waiting in queue...
The following packages have to be installed:
git-core-2.43.0-1.el8.x86_64    Core package of git with minimal functionality
Proceed with changes? [N/y] y

* Waiting in queue...
* Waiting for authentication...
* Waiting in queue...
* Downloading packages...
* Requesting data...
* Testing changes...
* Installing packages...
Cloning into 'k8s-demo'...
remote: Enumerating objects: 32, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (28/28), done.
remote: Total 32 (delta 2), reused 32 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (32/32), 11.19 KiB | 2.24 MiB/s, done.
Resolving deltas: 100% (2/2), done.

IOT1@IOT-ITI:~/project$ cd k8s-demo
IOT1@IOT-ITI:~/project/k8s-demo$ ls
backend  docker-compose.yml  frontend  k8s  LICENSE  migrator  README.md
IOT1@IOT-ITI:~/project/k8s-demo$ cd migrator
IOT1@IOT-ITI:~/project/k8s-demo/migrator$ ls
Dockerfile  migrate.py
```

```
IOT1@IOT-ITI:~/project/k8s-demo/k8s$ cd /
IOT1@IOT-ITI:~/project/k8s-demo$ ls -la backend/
total 12
drwxrwxr-x. 2 IOT1 IOT1 63 Aug 25 21:01 .
drwxrwxr-x. 7 IOT1 IOT1 152 Aug 25 21:01 ..
-rw-rw-r-- 1 IOT1 IOT1 289 Aug 25 21:01 Dockerfile
-rw-rw-r-- 1 IOT1 IOT1 2120 Aug 25 21:01 main.py
-rw-rw-r-- 1 IOT1 IOT1 80 Aug 25 21:01 requirements.txt
IOT1@IOT-ITI:~/project/k8s-demo$ ls -la frontend/
total 20
drwxrwxr-x. 2 IOT1 IOT1 81 Aug 25 21:01 .
drwxrwxr-x. 7 IOT1 IOT1 152 Aug 25 21:01 ..
-rw-rw-r-- 1 IOT1 IOT1 220 Aug 25 21:01 Dockerfile
-rw-rw-r-- 1 IOT1 IOT1 375 Aug 25 21:01 endpoint.sh
-rw-rw-r-- 1 IOT1 IOT1 7594 Aug 25 21:01 index.html
-rw-rw-r-- 1 IOT1 IOT1 191 Aug 25 21:01 nginx.conf
IOT1@IOT-ITI:~/project/k8s-demo$ ls -la migrator/
total 8
drwxrwxr-x. 2 IOT1 IOT1 42 Aug 25 21:01 .
drwxrwxr-x. 7 IOT1 IOT1 152 Aug 25 21:01 ..
-rw-rw-r-- 1 IOT1 IOT1 140 Aug 25 21:06 Dockerfile
-rw-rw-r-- 1 IOT1 IOT1 1548 Aug 25 21:04 migrate.py
IOT1@IOT-ITI:~/project/k8s-demo$ ls -la k8s/
total 24
drwxrwxr-x. 2 IOT1 IOT1 130 Aug 25 21:01 .
drwxrwxr-x. 7 IOT1 IOT1 152 Aug 25 21:01 ..
-rw-rw-r-- 1 IOT1 IOT1 1824 Aug 25 21:01 backend.yaml
-rw-rw-r-- 1 IOT1 IOT1 347 Aug 25 21:01 configmap.yaml
-rw-rw-r-- 1 IOT1 IOT1 781 Aug 25 21:01 frontend.yaml
-rw-rw-r-- 1 IOT1 IOT1 1183 Aug 25 21:01 migrator.yaml
-rw-rw-r-- 1 IOT1 IOT1 1549 Aug 25 21:01 postgres.yaml
-rw-rw-r-- 1 IOT1 IOT1 265 Aug 25 21:01 secret.yaml
IOT1@IOT-ITI:~/project/k8s-demo$ docker images | grep k8s-demo
k8s-demo/backend:latest          5faa859944fa          291MB          70.3MB
k8s-demo/frontend:latest        e10e2b8f67d           92MB           26.3MB
k8s-demo/migrator:latest        cd1b00709961          211MB          52.0MB
```

Figure 1: Verification of cloning

3 Docker Image Build and Push

3.1 Initial Image Building

Before pushing to the cloud, the images are initially built locally in their respective directories to verify the Dockerfiles work.

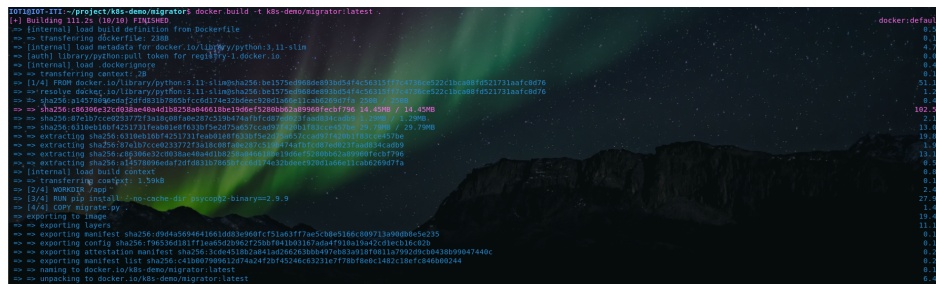


Figure 2: Building the migrator image locally.

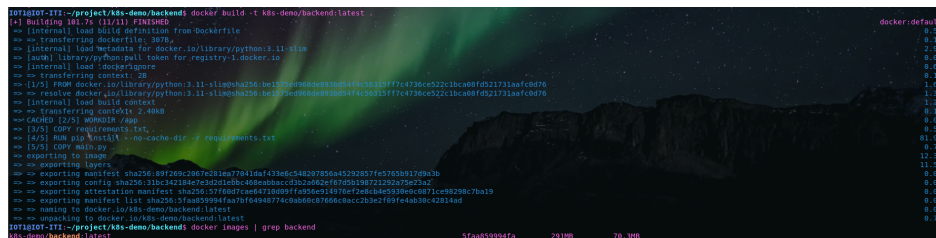


Figure 3: Building the backend image locally.

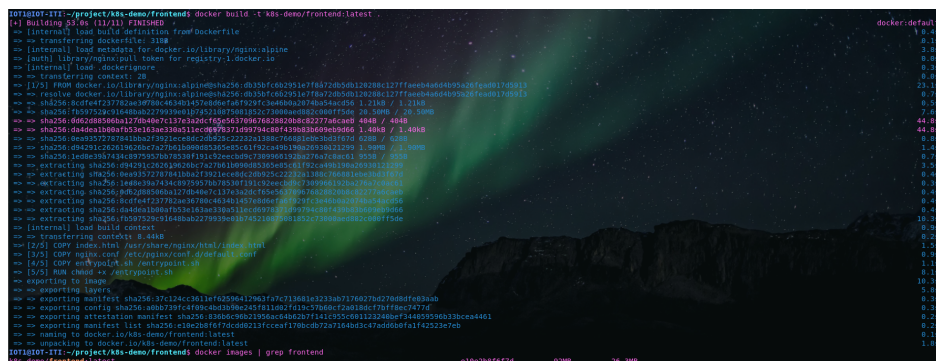


Figure 4: Building the frontend image locally.

3.2 Final Image Build for Push

The images are then rebuilt with the correct Docker Hub username tag (gannahossam860) so they can be pushed to the repository.

```
IoT1@IoT-ITI:~/project/k8s-demo$ docker build -t gannahossam860/k8s-demo-backend:latest ./backend
[+] Building 0.3s (1/2)
[+] Building 3.2s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 307B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:be1575ed968de893bd54f4c56315ff7c4736ce522c1bca08fd521731aafc0d76
=> => resolve docker.io/library/python:3.11-slim@sha256:be1575ed968de893bd54f4c56315ff7c4736ce522c1bca08fd521731aafc0d76
=> [internal] load build context
=> => transferring context: 182B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY main.py
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:89f169c2067e201ea77841daf433a6c548207856a45292857fe5765b91749a3b
=> => exporting config sha256:31bc342184e7e3d2d1ebbc468abbc0d3b2a662ef67d5b198721792a75e23a2
=> => exporting attestation manifest sha256:2d5c5de0bfc9de2b5f26577bf6a26679131eb7c4789f1939622d1055d889433
=> => exporting manifest list sha256:c475d596b0f96ce405f8624a57ac2b5527dfe962417b6852c39a85bdcad1082
=> => naming to docker.io/gannahossam860/k8s-demo-backend:latest
=> => unpacking to docker.io/gannahossam860/k8s-demo-backend:latest
```

Figure 5: Final build of the backend image for Docker Hub.

```
IoT1@IoT-ITI:~/project/k8s-demo$ docker build -t gannahossam860/k8s-demo-frontend:latest ./frontend
[+] Building 3.5s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 318B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913
=> => resolve docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913
=> [internal] load build context
=> => transferring context: 272B
=> CACHED [2/5] COPY index.html /usr/share/nginx/html/index.html
=> CACHED [3/5] COPY nginx.conf /etc/nginx/conf.d/default.conf
=> CACHED [4/5] COPY entrypoint.sh /entrypoint.sh
=> CACHED [5/5] RUN chmod +x /entrypoint.sh
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:37c124cc3611ef62596412963fa7c713681e3233ab7176027bd270d8dfe03aab
=> => exporting config sha256:a0bb739fc4f09c4bd3b90e245f811d02fd19c57b60cf2a018dcf7bfff8ec7477d
=> => exporting attestation manifest sha256:cfa183420a35ef1d889f4c22768393bb94d5b525ede92b4d6d439f8ec0c602e
=> => exporting manifest list sha256:3bd4336641ef4540e586648bc77592ff34872a321468c3e68517bc13984afd77
=> => naming to docker.io/gannahossam860/k8s-demo-frontend:latest
=> => unpacking to docker.io/gannahossam860/k8s-demo-frontend:latest
```

Figure 6: Final build of the frontend image for Docker Hub.

```
IoT1@IoT-ITI:~/project/k8s-demo$ docker build -t gannahossam860/k8s-demo-migrator:latest ./migrator
[+] Building 2.1s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 238B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/python:3.11-slim@sha256:be1575ed968de893bd54f4c56315ff7c4736ce522c1bca08fd521731aafc0d76
=> => resolve docker.io/library/python:3.11-slim@sha256:be1575ed968de893bd54f4c56315ff7c4736ce522c1bca08fd521731aafc0d76
=> [internal] load build context
=> => transferring context: 91B
=> CACHED [2/4] WORKDIR /app
=> CACHED [3/4] RUN pip install --no-cache-dir psycopg2-binary==2.9.9
=> CACHED [4/4] COPY migrate.py
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:d9d4a5694641661dd83e960fcf51a63ff7ae5cb8e5166c809713a90db8e5e235
=> => exporting config sha256:f96536d181ff1ea65d2b962f25bbf041b03167ada4f910a19a42cd1ecb16c02b
=> => exporting attestation manifest sha256:0428cc5228937fe8c6ac9fe0324b5702024558688c3cb60e74d7f15df6f8673
=> => exporting manifest list sha256:767a0242f60fbfaa6591c8889a343fe799be911be723d9c4ff98997f1dd097b7
=> => naming to docker.io/gannahossam860/k8s-demo-migrator:latest
=> => unpacking to docker.io/gannahossam860/k8s-demo-migrator:latest
```

Figure 7: Final build of the migrator image for Docker Hub.

3.3 Pushing to Docker Hub

The images are tagged and pushed to Docker Hub under the user `gannahossam860`.

```
IOT1@IOT-ITI:~/project/k8s-demo$ docker login
Authenticating with existing credentials... [Username: gannahossam860]

Info -> To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
IOT1@IOT-ITI:~/project/k8s-demo$ docker push gannahossam860/k8s-demo-backend:latest
The push refers to repository [docker.io/gannahossam860/k8s-demo-backend]
8a5ca0a62e8c: Pushed
44136fa355b3: Already exists
87e1b7cce023: Layer already exists
6310eb16bf42: Layer already exists
c86306e32cd0: Layer already exists
a14578096eda: Layer already exists
90a411fb54b1: Layer already exists
8cb27590dd0e: Layer already exists
a8dc99674c68: Layer already exists
cee5aa6eabfa: Layer already exists
latest: digest: sha256:c475d596b0f96ce405f8624a57ac2b5527df8e962417b6852c39a85bdcad1082 size: 856
```

Figure 8: Logging in and pushing the backend image.

```
IOT1@IOT-ITI:~/project/k8s-demo$ docker push gannahossam860/k8s-demo-frontend:latest
The push refers to repository [docker.io/gannahossam860/k8s-demo-frontend]
44136fa355b3: Already exists
d94291c26261: Layer already exists
55afa1ecc21d: Layer already exists
7d9a9d8d5e52: Pushed
0ea935727878: Layer already exists
1ed8e39a7434: Layer already exists
0d62d88506ba: Layer already exists
8cdfef4f23778: Layer already exists
da4deab00af: Layer already exists
2714281a2c2e: Layer already exists
f48c8fdff0a19: Layer already exists
fb597529c916: Layer already exists
4abe0ab3fac7: Layer already exists
402383e5cbc0: Layer already exists
latest: digest: sha256:3bd4336641ef4540e586648bc77502ff34872a321468c3e60517bc13984afd77 size: 856
IOT1@IOT-ITI:~/project/k8s-demo$ docker push gannahossam860/k8s-demo-migrator:latest
The push refers to repository [docker.io/gannahossam860/k8s-demo-migrator]
020058940285: Pushed
44136fa355b3: Already exists
87e1b7cce023: Layer already exists
6310eb16bf42: Layer already exists
c86306e32cd0: Layer already exists
a14578096eda: Layer already exists
90a411fb54b1: Layer already exists
aa79e75b2a14: Layer already exists
1c4645b8b8dd: Layer already exists
latest: digest: sha256:767a0242f60fbfaa6591c8889a343fe799be911be723d9c4ff98997f1dd097b7 size: 856
```

Figure 9: Pushing the frontend and migrator images.

Name	Last Pushed 	Contains	Visibility	Scout
gannahossam860/k8s-demo-migrator	about 18 hours ago		Public	Inactive
gannahossam860/k8s-demo-frontend	about 18 hours ago		Public	Inactive
gannahossam860/k8s-demo-backend	about 18 hours ago		Public	Inactive

Figure 10: Verification of the pushed images on Docker Hub.

4 Docker Run Execution

The containers are run manually using `docker run` and an isolated network. The frontend is accessed locally to verify the connection.

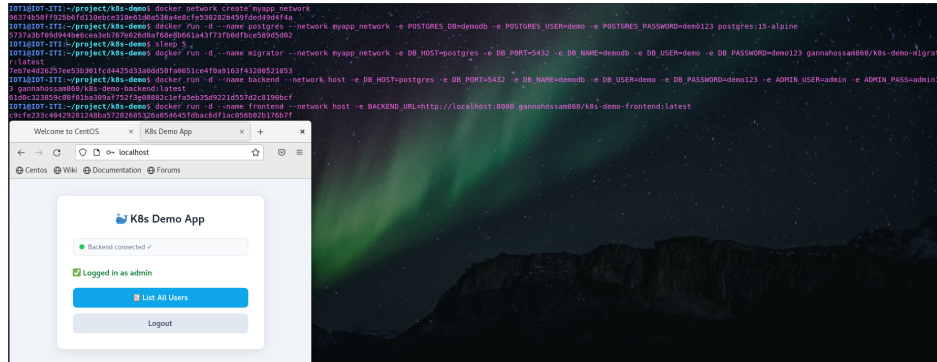


Figure 11: Running all services using `docker run` and verifying the login.

5 Docker Compose Execution

The `docker-compose.yml` file is used to orchestrate the entire stack with the correct dependencies.

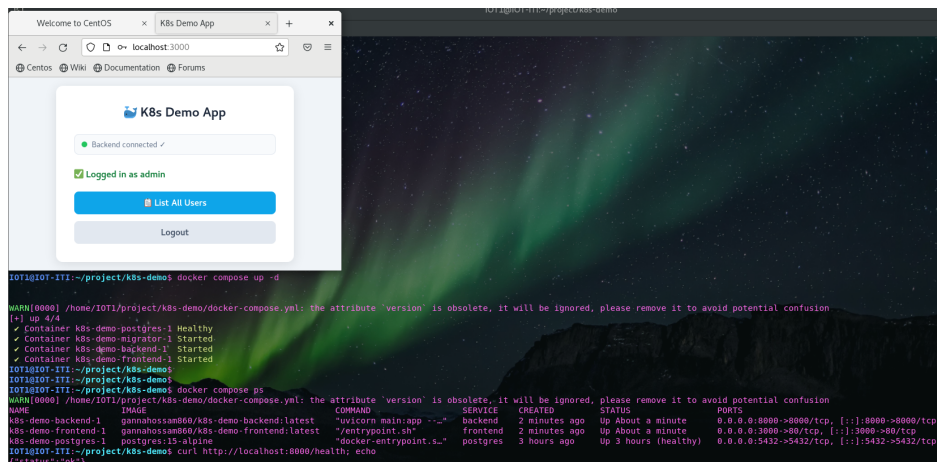


Figure 12: Running the stack with `docker compose up -d` and verifying health status.

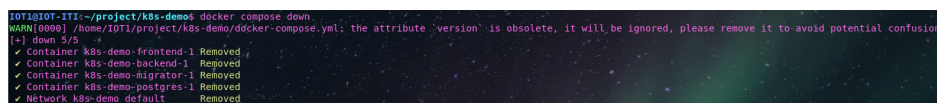


Figure 13: Tearing down the stack with `docker compose down`.

6 Kubernetes Run Execution

6.1 Building Images for Minikube

Docker images are rebuilt without cache to ensure they match the local Minikube environment.

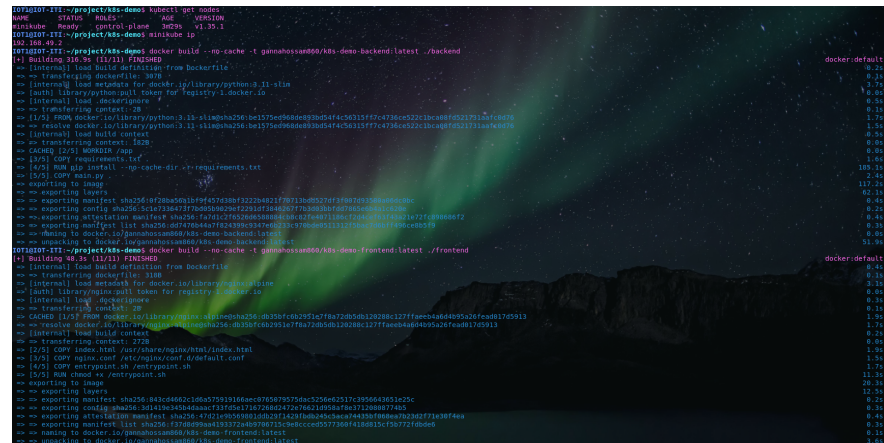


Figure 14: Building the backend and frontend images for Kubernetes.

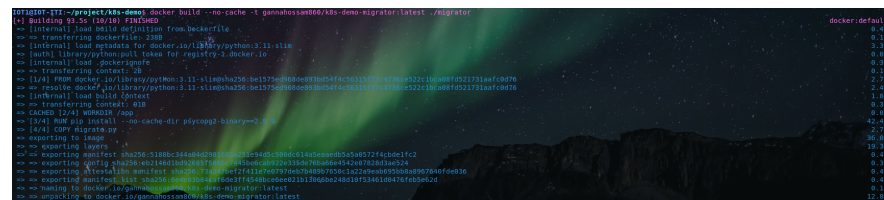


Figure 15: Building the migrator image for Kubernetes.

6.2 Verifying for Minikube

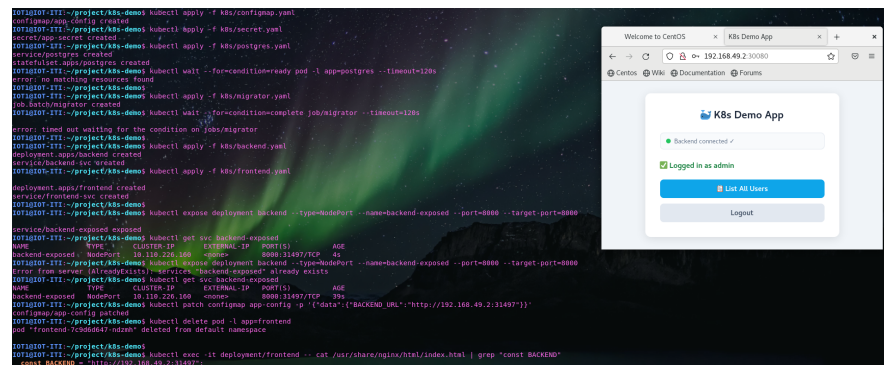


Figure 16: running in Kubernetes and logging in.

7 Final Output

Regardless of the method used, the application achieves the same final output after successfully logging in. The following screenshot represents the successfully complete 3-tier application showing the data retrieved from the database.

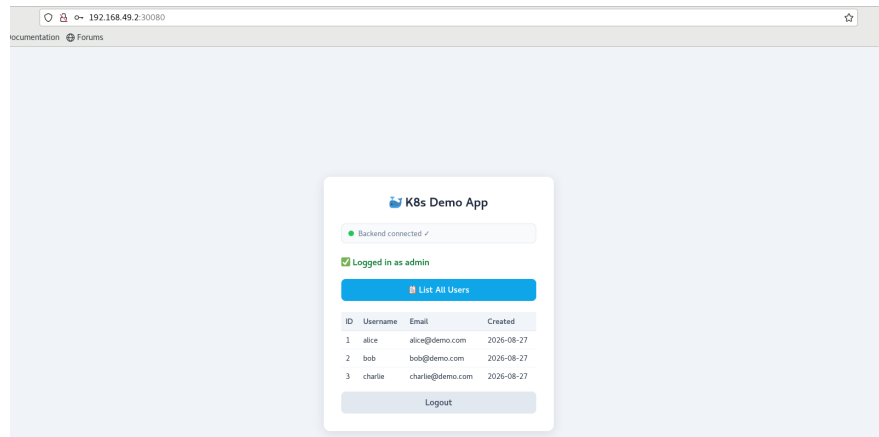


Figure 17: Final output achieved by all three methods after login.

8 Architecture Diagram

The following diagram illustrates the complete architecture of the 3-Tier application across the three environments: Docker Run, Docker Compose, and Kubernetes (Minikube).

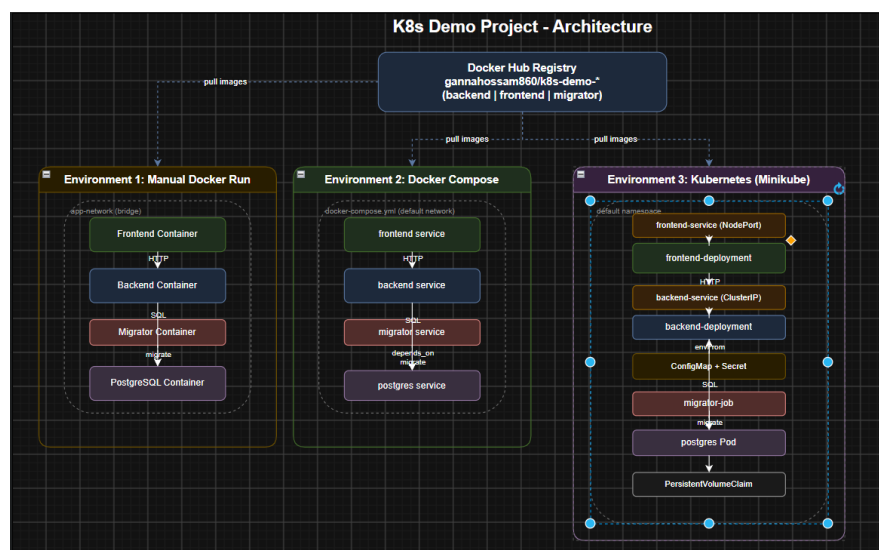


Figure 18: Complete architecture of the K8s Demo Project.

9 Conclusion

This Project successfully demonstrated the building of a full-stack application across three different container environments: manual Docker Run, Docker Compose, and Kubernetes. The project highlighted the critical concepts of runtime configuration injection, container networking, service discovery, and production-grade deployment practices.